

SchemaForge v2.0

Autonomous Database Migration Reliability Agent

Evidence-backed migration rehearsal before production change

MISSION

Allow an agent to investigate and rehearse consequential database changes without giving the agent blind production authority.

CORE STACK

TrueForge • MCP • PostgreSQL 16 • Isolated sandbox / shadow clone • Human approval

SAFETY PRINCIPLE

The agent may reason and test autonomously. Production mutation remains a separate, action-bound, human-authorized capability.

Engineering revision of the original SchemaForge specification. This version incorporates the technical judge concerns identified during review: realistic shadow data, measured-vs-predicted risk, PostgreSQL transaction semantics, application compatibility, schema-drift protection, action-bound approvals, and safer rollback language.

Prepared for the TrueFoundry × Polaris "Agents That Act" Hackathon • 25 September 2026

1. Executive Summary

What SchemaForge is and why it exists

SchemaForge is not an SQL generator. It is an agentic database-change control system. A human gives a natural-language migration objective; TrueForge orchestrates schema inspection, migration synthesis, sandbox rehearsal, verification, risk reporting, and a hard approval boundary before a production mutation.

The original design correctly centered on the sequence **Real System** → **Generated Code** → **Sandbox** → **Human Control** → **TrueForge**. This revision strengthens that sequence by replacing the weak “empty sandbox + happy-path migration” story with a realistic shadow/rehearsal environment and an explicit refusal path.

The product is aimed at mission-critical PostgreSQL environments where database changes are stateful, potentially expensive, and difficult to reverse. The goal is to automate the tedious verification work while preserving human authority over consequential production changes.

Core workflow

1	Intent	Human states desired database change
2	Inspect	Agent reads live schema, constraints, sizes, dependencies
3	Plan	Agent generates forward migration + rollback where reversible
4	Rehearse	Migration runs against realistic shadow data / seeded failure cases
5	Verify	Generated and deterministic checks produce evidence
6	Decide	Agent recommends APPLY / REVIEW / DO NOT APPLY
7	Gate	TrueForge freezes before consequential production action
8	Execute	Only the approved, action-bound migration is applied
9	Verify	Post-apply read-only checks confirm the intended state

2. Judging Rubric Engineering

Design directly against the official five-criterion structure

Criterion	Weight	How SchemaForge demonstrates it
The Harness is Doing the Work	30%	TrueForge owns the execution lifecycle, MCP orchestration, sandbox invocation, state persistence, and approval
It Actually Runs	25%	Real PostgreSQL 16 target, real MCP server, real sandbox execution, real generated migration, real approval
Where It Stops	20%	The central boundary: after evidence is produced but before production mutation. TrueForge freezes execution
A Job Worth Handing Over	15%	DBA/SRE verification work is repetitive and high consequence. SchemaForge automates inspection and rehearsal
Demo Clarity	10%	One linear story: request → inspect → detect risk → rehearse → evidence → stop → approve → apply → verify

Design principle: maximize the score by making TrueForge visibly necessary. The agent should not merely generate SQL; it should inspect a real system, execute generated verification code in a sandbox, stop at the consequential boundary, and resume from preserved state after approval.

What changed from v1

- **Realistic rehearsal:** replace the empty sandbox as the primary proof with a shadow clone / representative dataset and intentionally seeded edge cases.
- **Safer execution:** production write credentials are isolated from the normal agent loop; approval is bound to one exact migration.
- **Post-inspection drift check:** re-hash or re-read the target schema immediately before apply.
- **PostgreSQL correctness:** classify DDL by transactional semantics; do not assume every migration belongs inside one transaction.
- **Evidence discipline:** observed lock/latency measurements are labeled as observed; estimates are labeled as estimates.
- **Rollback discipline:** rollback is generated and rehearsed where reversible; irreversible/destructive operations are prohibited or escalated.
- **Application compatibility:** inspect repository references/dependent database objects where possible.

3. End-to-End Workflow

The agent lifecycle from intent to verified production state

Stage 1 — Intent ingestion. The human describes the desired change in plain English. Example: “Make users.email NOT NULL.”

Stage 2 — Live introspection. SchemaForge calls `db_inspect_schema` and collects columns, data types, nullability, indexes, foreign keys, constraints, relation size, row counts, and relevant database metadata.

Stage 3 — Dependency and compatibility analysis. The agent checks dependent database objects and, where a repository is available, searches application code for references to affected tables/columns.

Stage 4 — Migration synthesis. The agent produces forward DDL, a rollback plan when the operation is genuinely reversible, and a deterministic verification plan.

Stage 5 — Shadow rehearsal. The migration is tested against a realistic PostgreSQL shadow environment populated with representative data and, for the demo, controlled edge cases. The rehearsal may include row counts, constraint checks, query plans, lock behavior, and data-integrity checks.

Stage 6 — Decision packet. The system produces evidence, not just an LLM opinion: exact DDL, affected objects, detected violations, measured timings, verification results, rollback status, and schema fingerprint.

Stage 7 — Consequential gate. TrueForge freezes the agent loop before the production mutation. The approval action is bound to the exact migration hash and target.

Stage 8 — Production execution. The gated executor applies only the approved migration using the appropriate PostgreSQL execution mode.

Stage 9 — Post-apply verification. Read-only checks confirm that the intended schema/state exists and that key invariants remain valid.

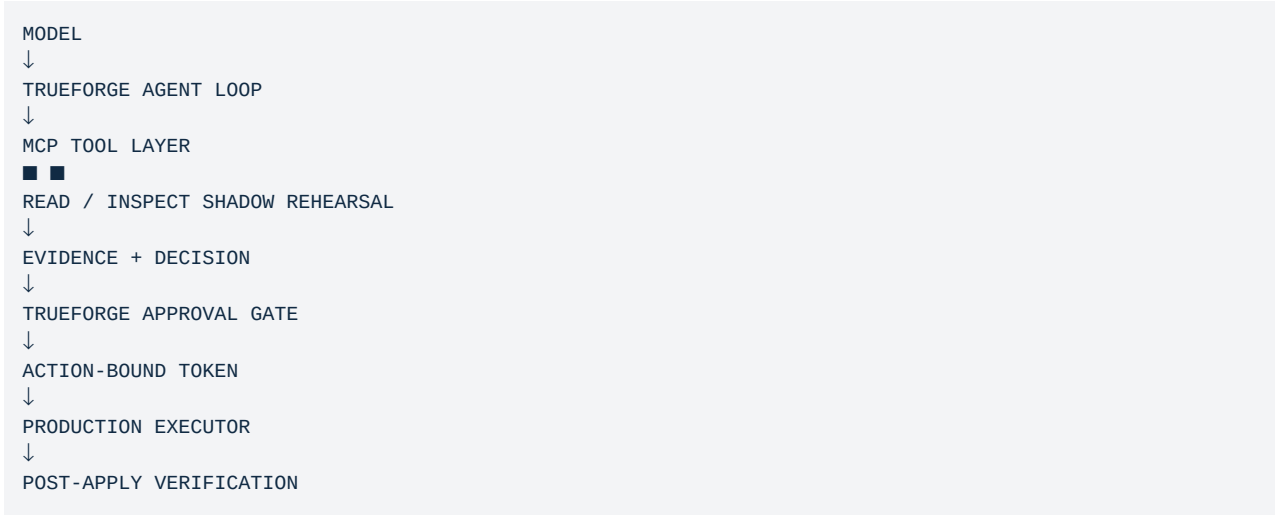
The key behavioral distinction

A trustworthy agent is not one that always acts. It is one that can gather enough evidence to act, and can also produce a defensible DO NOT APPLY decision when the rehearsal reveals a problem.

4. Architecture & Security Boundaries

Model, harness, tools, sandbox, executor, and human authority are separate

MODEL	Reasoning, migration synthesis, hypothesis generation, risk explanation
TRUEFORGE HARNESS	Agent lifecycle, MCP dispatch, sandbox invocation, session state, approval interlock
MCP TOOL SERVER	Structured database inspection, read-only queries, rehearsal operations, gated executor
SHADOW / SANDBOX	Isolated PostgreSQL rehearsal with representative data; no production write credentials
PRODUCTION EXECUTOR	Dedicated component with write capability; invoked only after valid approval
HUMAN	Reviews evidence and authorizes the exact consequential action



Critical credential rule

The normal agent loop should not possess unrestricted production write credentials. The write capability belongs to the gated executor. Approval should authorize one exact migration, for one target, for a limited time, and only once.

5. Codebase & Tool Blueprint

Minimal implementation that still demonstrates real agentic execution

```
schemaforge/
├── docker-compose.yml
├── .env.example
├── README.md
├── mcp-server/
│   ├── package.json
│   ├── tsconfig.json
│   └── src/
│       ├── index.ts
│       ├── db.ts
│       └── tools/
│           ├── inspect-schema.ts
│           ├── run-query.ts
│           ├── analyze-dependencies.ts
│           ├── rehearse-migration.ts
│           └── execute-migration.ts
├── shadow/
│   ├── seed-schema.sql
│   ├── seed-data.sql
│   ├── seed-edge-cases.sql
│   ├── validate-migration.sh
│   └── verification/
│       ├── schema-fingerprint.ts
│       ├── invariant-checks.ts
│       └── post-apply.ts
├── trueforge-config/
│   ├── agent.yaml
│   └── system-prompt.md
```

Tool responsibilities

db_inspect_schema	Read structured PostgreSQL catalog metadata. No mutation.
db_run_readonly_query	Allowlisted/read-only analysis with statement timeout and result limits.
analyze_dependencies	Identify FKs, indexes, views/triggers where available, and repository references when configured.
rehearse_migration	Run generated migration and verification plan against the shadow environment.
execute_migration	Consequential tool. Available only through TrueForge approval and an action-bound authorization.
verify_production	Read-only post-apply checks and schema fingerprint comparison.

Read-only security

Use a database role restricted to SELECT for inspection. Enforce statement timeouts and result-size limits. Do not treat a regex alone as the security boundary; use database permissions, tool-level validation, explicit allowlists where feasible, and the separate production executor.

6. Safety Engineering

Four operational tiers plus action-bound authorization

Tier	Examples	Governance
0 — Safe autonomous	Schema inspection, read-only analysis, sandbox execution	Agent may act automatically. No production state change.
1 — Local / reversible	Generate DDL, generate rollback, rehearsal, deterministic simulation	Local simulation side effects only. Audit everything.
2 — Consequential	Targeted production DDL	Strict TrueForge human approval. Exact action must match approval
3 — Prohibited	DROP TABLE, TRUNCATE, role/permission changes, destructive data layer	Respective data layer; do not expose a production p

Approval token model

```
approval_token = {
  migration_hash: "sha256:...",
  target: "public.users",
  action: "ALTER COLUMN email SET NOT NULL",
  created_at: "...",
  expires_at: "...",
  single_use: true
}

execute_migration(token, migration):
  verify token is valid
  verify migration hash matches
  verify target matches
  verify schema fingerprint has not drifted
  execute using correct PostgreSQL mode
  invalidate token
```

Default behavior

- **No pre-selected approval.** The safe default is review/reject.
- **Failed verification → DO NOT APPLY.** The UI should not offer a misleading “approve anyway” path for a failed safety check.
- **Schema drift → re-run rehearsal.** If the target changes after inspection, the old approval becomes invalid.

7. Realistic Shadow Rehearsal

The major improvement over the original empty-container approach

The original specification used a fresh empty PostgreSQL instance seeded with schema and sample DML. That is useful for proving syntax, but it is not enough to prove production behavior. The revised design uses a representative shadow environment for the core safety demonstration.

Evidence	What we want to prove
Schema compatibility	The DDL compiles and applies to the relevant PostgreSQL version.
Data compatibility	Existing representative rows satisfy the proposed constraints and transformations.
Constraint integrity	FK, CHECK, UNIQUE and NOT NULL invariants remain valid.
Query behavior	Important query plans / index usage behave as intended.
Operational behavior	Where measurable, record observed execution time and lock behavior during rehearsal.
Rollback / recovery	Reversible changes return to the expected baseline; destructive changes are not treated as automatically reversible
Application compatibility	Search for references to affected schema objects and flag obvious breaking changes.
Schema fingerprint	Capture the inspected state so a stale rehearsal cannot silently authorize a changed database.

Demo dataset strategy

- Use a realistic e-commerce schema: users, products, orders, order_items.
- Populate enough rows to make validation meaningful for the demo.
- Seed deliberate edge cases: e.g. a small number of NULL emails for a NOT NULL migration.
- Keep the scenario deterministic so the same request produces the same evidence every rehearsal.

8. Five-Minute Judge Demo

The demo should show both refusal and successful controlled execution

Time	What the judge sees	Message
0:00–0:45	Problem + architecture	"Database changes are stateful. SchemaForge lets an agent do the verification work without
0:45–2:00	Dangerous request	Request: "Make users.email NOT NULL." Agent inspects the live target and identifies the rel
2:00–3:00	Shadow rehearsal	Migration runs against realistic shadow data. Verification discovers 14 NULL emails. Decisio
3:00–3:40	Corrected request	Request is changed to a safe migration, e.g. add archived_at as nullable.
3:40–4:20	Approval boundary	6+ checks pass. Evidence packet appears. TrueForge freezes execution. No approval is pre
4:20–5:00	Real execution + proof	Human approves the exact migration. Production executor applies it. Read-only verification

The centerpiece screen

```
MIGRATION DECISION PACKET

Target: public.users
Requested change: email → NOT NULL

Rows inspected:      182,340
Violations found:    14
Shadow rehearsal:    FAILED
Rollback rehearsal:  VERIFIED
Schema drift:        NONE
Application references: 3 flagged

DECISION:  DO NOT APPLY

Production database: UNTOUCHED
```

9. Judge Attack Questions & Answers

Answers the technical questions most likely to expose weak assumptions

Q: Why AI instead of Flyway/Liquibase?

A: SchemaForge does not replace the migration engine. It adds an adaptive inspection, rehearsal, evidence, and decision layer around a requested change. Static migration tooling executes known migrations; SchemaForge reasons from the current target state and generates verification around the specific change.

Q: Why should I trust an empty sandbox?

A: We do not rely on an empty sandbox as our main proof. The revised design uses a representative shadow dataset and controlled edge cases. We distinguish syntax validation from data/operational evidence.

Q: How do you know the lock duration?

A: Observed timings come from the rehearsal/execution instrumentation. Any production value not directly measured is labeled an estimate or risk category, not presented as fact.

Q: What prevents the agent from running DROP TABLE?

A: Multiple layers: prohibited operation policy, tool validation, read-only database role for inspection, no unrestricted write credential in the agent loop, and a separate gated executor. The database itself remains a security boundary.

Q: Can one approval authorize several migrations?

A: No. The authorization is action-bound to a migration hash, target, expiry, and single-use state.

Q: What if the database changes after rehearsal?

A: A schema fingerprint is captured during inspection/rehearsal. The executor verifies it before applying. Drift invalidates the authorization and forces a new rehearsal.

Q: What if rollback is impossible?

A: We do not claim universal reversibility. Irreversible/destructive operations are prohibited or escalated. Rollback is generated and rehearsed only when the operation is meaningfully reversible.

Q: What if the database change succeeds but the application breaks?

A: We include dependency/application-reference analysis where the repository is available and explicitly flag breaking schema changes. Post-apply verification checks database invariants; application-level rollout remains a separate deployment concern.

Q: What makes TrueForge necessary?

A: TrueForge is the execution authority: it manages the agent lifecycle, MCP tool calls, sandbox execution, preserved state, and the consequential approval barrier. The demo should visibly depend on these capabilities.

Q: Is this a real production database?

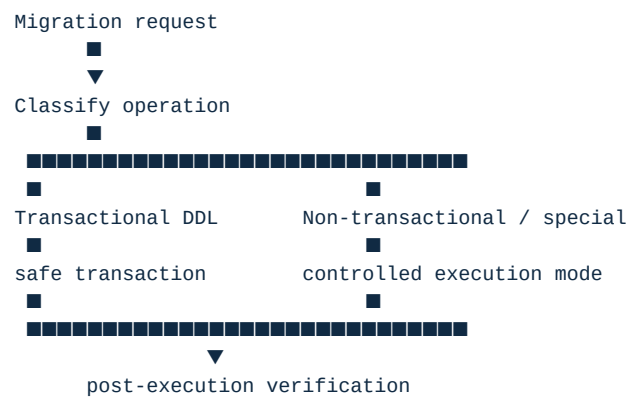
A: For the hackathon, it is a controlled PostgreSQL environment representing the target system. The important point is that the database engine, connection roles, migration execution, sandbox, and verification are real—not mocked API responses.

10. PostgreSQL Correctness Rules

Specific implementation traps to avoid during the build

- Rule 1 — Do not treat every DDL statement as transaction-safe.** PostgreSQL operations have different transactional semantics. For example, `CREATE INDEX CONCURRENTLY` cannot be executed inside a transaction block. The migration planner must classify operations and choose the appropriate execution strategy.
- Rule 2 — Never call an LLM estimate a measurement.** If the system says “47 ms,” the UI must indicate whether 47 ms was observed during rehearsal or inferred. Prefer measured evidence.
- Rule 3 — Rollback is not magic.** A dropped column or destructive data transformation may not be reconstructable. The system must distinguish reversible schema changes from irreversible data loss.
- Rule 4 — Regex is not the security boundary.** Read-query filtering is useful, but actual database permissions, statement timeouts, tool constraints, and execution isolation are stronger controls.
- Rule 5 — Re-check before apply.** A migration approved against yesterday’s schema must not silently apply to a changed target. Bind approval to the inspected state.

Recommended execution classifier



11. Seven-Hour Build Plan

Scope discipline is a feature

Hour	Agent / orchestration	MCP / DB	Sandbox / verification	Demo / safety
0–1	TrueForge boot + model/API	PostgreSQL + MCP skeleton	Shadow DB skeleton	Repo + README
1–2	Prompt + lifecycle	Inspect schema + read-only tool	Seed realistic data	Basic demo shell
2–3	Tool orchestration	Migration synthesis	Rehearsal runner	Failure scenario
3–4	Decision logic	Execution tool	Constraint/data checks	Approval gate
4–5	Evidence packet	Schema fingerprint	Rollback/recovery checks	Action-bound approval
5–6	End-to-end rehearsal	Fix DB edge cases	Deterministic reset	Security audit
6–7	Freeze features	Final build	Full rehearsal	Backup video + final run

Non-negotiable scope cuts

- PostgreSQL only.
- One controlled e-commerce schema.
- One dangerous migration scenario + one safe migration scenario.
- No multi-database support, Kubernetes, Slack/Jira integrations, authentication system, or generic multi-agent swarm.
- If a feature does not improve the five judging criteria or demo reliability, do not build it.

Definition of done

A judge can watch a real TrueForge session inspect a real PostgreSQL target, generate a real migration, run real generated verification in an isolated rehearsal environment, see a defensible DO NOT APPLY decision for a failing scenario, then approve a separate safe migration and watch the real target change and verify successfully.

12. Real-World Use & Product Direction

What exists beyond the hackathon demo

Primary users: software teams, SREs, DBAs, platform engineering teams, fintech/healthcare/e-commerce engineering groups, and startups where developers share database-operations responsibility.

Real workflow: a developer opens a pull request containing a migration. SchemaForge can inspect the target environment, rehearse the change, report violations and dependencies, and return an evidence-backed status such as READY FOR REVIEW or DO NOT APPLY. A DBA or authorized engineer remains the final authority for consequential production changes.

Business value: reduce manual migration-review effort, catch data/constraint problems before production, create an audit trail for change decisions, and reduce the chance of high-cost database incidents.

Product direction: SchemaForge could eventually integrate with CI/CD and existing migration systems rather than replacing them. Its differentiator is the adaptive evidence and decision layer around database changes.

What the product is not

- Not a generic SQL chatbot.
- Not a replacement for PostgreSQL expertise.
- Not an autonomous production DBA with unrestricted credentials.
- Not a guarantee that every migration is reversible or safe.

Final positioning

SchemaForge gives an AI the ability to investigate and rehearse database changes without giving the AI blind authority over the database. It converts a natural-language request into an evidence-backed migration decision, stops at the consequential boundary, and only resumes when a human authorizes the exact action.

Team presentation line

“We are not trying to make the AI brave enough to change production. We are trying to make it evidence-driven enough to know when it should—and when it absolutely should not.”

Revision notes: This v2.0 incorporates the engineering corrections identified during technical review of the original 10-page SchemaForge specification. The original specification established the core TrueForge/MCP/PostgreSQL workflow, four-tier safety model, six-check validation concept, approval interlock, and five-minute demo structure; this revision strengthens those components rather than changing the project's core identity.